

05_model-rule-py

#SQLAlchemy

#PostgreSQL

#python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ models/  
            └─ rule.py
```

El archivo `rule.py` se encuentra dentro de la carpeta de modelos del backend:

```
backend/app/models/
```

Este archivo define el modelo SQLAlchemy `Rule`, que representa la tabla de reglas del laboratorio SIEM MVP.

Una regla es una condición configurada por el sistema para decidir si uno o varios eventos deben generar una alerta.

Dentro del flujo del laboratorio, el modelo `Rule` se sitúa entre los eventos recibidos y las alertas generadas:

```
Evento recibido  
    ↓  
Consulta de reglas activas  
    ↓  
Evaluación de criterios  
    ↓  
Coincidencia con una Rule  
    ↓  
Generación de Alert
```

Este modelo es importante porque define qué criterios puede usar el laboratorio para detectar eventos relevantes.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' backend/app/models/rule.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

```
backend/app/models/rule.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from typing import Any, Optional
from sqlalchemy import Boolean, DateTime, Integer, String, func
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base

class Rule(Base):
    __tablename__ = "rules"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False, unique=True)
    enabled: Mapped[bool] = mapped_column(Boolean, nullable=False, server_default="true")

    # criterios
    source: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    severity_min: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    contains: Mapped[Optional[str]] = mapped_column(String(200), nullable=True)

    # throttle por regla (segundos)
    throttle_seconds: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

    # threshold: N eventos en X segundos
    threshold_count: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    threshold_seconds: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

    # match por meta
    meta_match: Mapped[Optional[dict[str, Any]]] = mapped_column(JSONB, nullable=True)

    created_at: Mapped[object] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )
```

4 Función general del archivo

El archivo `rule.py` define la estructura de las reglas de detección dentro de la base de datos.

La clase principal es:

```
class Rule(Base):
```

Esta clase hereda de `Base`, la clase declarativa definida en:

```
backend/app/db/base.py
```

Gracias a esta herencia, SQLAlchemy interpreta `Rule` como un modelo ORM.

La tabla asociada se llama:

```
__tablename__ = "rules"
```

Por tanto, la relación es:

```
Clase Python Rule
↓
Tabla PostgreSQL rules
```

Este modelo permite almacenar reglas con diferentes criterios de detección:

```
source          → origen del evento
severity_min    → severidad mínima requerida
contains        → texto que debe aparecer en el mensaje
throttle_seconds → tiempo mínimo entre alertas de la misma regla
threshold_count → número de eventos requeridos
threshold_seconds → ventana temporal del threshold
meta_match      → coincidencias sobre el campo meta del evento
enabled         → indica si la regla está activa
```

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
from typing import Any, Optional
```

Importaciones de tipos.

```
from sqlalchemy import Boolean, DateTime, Integer, String, func
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column
```

Importaciones de SQLAlchemy.

```
from app.db.base import Base
```

Importación de la clase base declarativa.

```
class Rule(Base):
    __tablename__ = "rules"
```

Definición del modelo y nombre de la tabla.

```
id
name
enabled
source
severity_min
contains
throttle_seconds
threshold_count
threshold_seconds
meta_match
created_at
```

Definición de columnas.

Visualmente:

```
rule.py
├─ Importaciones de typing
├─ Importaciones SQLAlchemy
├─ Importación de Base
└─ Modelo Rule
    ├─ __tablename__
    ├─ id
    ├─ name
    ├─ enabled
    ├─ source
    ├─ severity_min
    └─ contains
```

```
| throttle_seconds
| threshold_count
| threshold_seconds
| meta_match
| created_at
```

6 Análisis línea por línea

Importación de Any y Optional

```
from typing import Any, Optional
```

Esta línea importa dos tipos desde el módulo estándar `typing`.

Any

Representa cualquier tipo de dato.

En este archivo se usa aquí:

```
dict[str, Any]
```

Esto permite que el diccionario `meta_match` tenga claves de tipo texto y valores de cualquier tipo.

Optional

Indica que un valor puede ser del tipo indicado o puede ser `None`.

En este archivo se utiliza en varios campos:

```
Optional[str]
Optional[int]
Optional[dict[str, Any]]
```

Esto significa que esos campos no son obligatorios y pueden quedar vacíos en la base de datos.

Importación de tipos SQLAlchemy

```
from sqlalchemy import Boolean, DateTime, Integer, String, func
```

Esta línea importa varios tipos y utilidades de SQLAlchemy.

Boolean

Boolean

Representa una columna booleana.

Se usa en:

```
enabled
```

Este campo indica si una regla está activa o no.

DateTime

```
DateTime
```

Representa una columna de fecha y hora.

Se usa en:

```
created_at
```

Integer

```
Integer
```

Representa una columna de número entero.

Se usa en:

```
severity_min  
throttle_seconds  
threshold_count  
threshold_seconds
```

String

```
String
```

Representa una cadena de texto con longitud limitada.

Se usa en:

```
name  
source  
contains
```

func

```
func
```

Permite usar funciones SQL desde SQLAlchemy.

En este archivo se usa:

```
func.now()
```

Esto representa la función SQL `NOW()` , utilizada para generar automáticamente la fecha de creación.

Importación de JSONB

```
from sqlalchemy.dialects.postgresql import JSONB
```

Esta línea importa el tipo `JSONB` , específico de PostgreSQL.

`JSONB` permite almacenar información JSON de forma eficiente.

En este archivo se usa para:

```
meta_match
```

Esto permite que una regla pueda definir condiciones sobre el campo `meta` de un evento.

Ejemplo conceptual:

```
{
  "user": "admin",
  "action": "login_failed"
}
```

Importación de `Mapped` y `mapped_column`

```
from sqlalchemy.orm import Mapped, mapped_column
```

Esta línea importa elementos del ORM moderno de SQLAlchemy.

`Mapped`

```
Mapped
```

Se usa para anotar atributos que representan columnas mapeadas por SQLAlchemy.

Ejemplo:

```
id: Mapped[int]
```

`mapped_column`

```
mapped_column
```

Se usa para definir las columnas de la tabla.

Ejemplo:

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Importación de `Base`

```
from app.db.base import Base
```

Esta línea importa la clase `Base` definida en:

```
backend/app/db/base.py
```

`Rule` hereda de esta clase:

```
class Rule(Base):
```

Gracias a esto, SQLAlchemy registra `Rule` como modelo ORM.

Definición de la clase `Rule`

```
class Rule(Base):
```

Esta línea define la clase `Rule`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
Rule
```

Nombre de la clase.

Representa una regla de detección del laboratorio SIEM.

```
(Base)
```

Indica que la clase hereda de `Base`.

Esto convierte `Rule` en un modelo SQLAlchemy.

Nombre de la tabla

```
__tablename__ = "rules"
```

Esta línea indica el nombre de la tabla asociada al modelo.

```
__tablename__
```

Es un atributo especial utilizado por SQLAlchemy.

```
"rules"
```

Es el nombre real de la tabla en PostgreSQL.

La relación es:

```
Rule → rules
```

Campo `id`

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Esta línea define la columna `id`.

Desglose:

```
id
```

Nombre del atributo y de la columna.

```
Mapped[int]
```

Indica que es una columna mapeada de tipo entero.

```
mapped_column(primary_key=True)
```

Define la columna como clave primaria.

La clave primaria identifica de forma única cada regla.

Campo `name`

```
name: Mapped[str] = mapped_column(String(120), nullable=False, unique=True)
```

Esta línea define la columna `name`.

Representa el nombre de la regla.

Desglose:

```
name
```

Nombre del atributo y de la columna.

```
Mapped[str]
```

Indica que el valor esperado es una cadena de texto.

```
String(120)
```

Define una cadena con longitud máxima de 120 caracteres.

```
nullable=False
```

Indica que el nombre es obligatorio.

```
unique=True
```

Indica que no puede haber dos reglas con el mismo nombre.

Esto evita duplicar reglas con nombres idénticos.

Ejemplo conceptual:

```
Multiple failed logins
High severity firewall event
Suspicious admin activity
```

Campo `enabled`

```
enabled: Mapped[bool] = mapped_column(Boolean, nullable=False, server_default="true")
```

Esta línea define la columna `enabled`.

Indica si la regla está activada o desactivada.

Desglose:

```
enabled
```

Nombre del atributo y de la columna.

```
Mapped[bool]
```

Indica que el valor esperado es booleano.

```
Boolean
```

Tipo SQL booleano.

```
nullable=False
```

El campo no puede quedar vacío.

```
server_default="true"
```

Indica que, si no se especifica un valor, la base de datos pondrá la regla como activa por defecto.

Esto significa que una regla nueva estará habilitada salvo que se indique lo contrario.

Comentario criterios

```
# criterios
```

Esta línea es un comentario.

En Python, los comentarios empiezan por `#` y no se ejecutan.

Este comentario separa visualmente los campos que definen las condiciones básicas de una regla.

Los criterios principales son:

```
source
severity_min
contains
```

Campo source

```
source: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
```

Esta línea define la columna `source`.

Permite limitar una regla a eventos de un origen concreto.

Desglose:

```
source
```

Nombre del campo.

```
Mapped[Optional[str]]
```

Indica que puede ser una cadena o `None`.

```
String(64)
```

Cadena con longitud máxima de 64 caracteres.

```
nullable=True
```

El campo puede ser nulo.

Ejemplo:

```
source = "auth"
```

La regla solo aplicaría a eventos cuyo origen sea `auth`.

Si `source` es `None`, la regla no filtra por origen.

Campo severity_min

```
severity_min: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
```

Esta línea define la columna `severity_min`.

Permite indicar una severidad mínima para que la regla se cumpla.

Desglose:

```
severity_min
```

Nombre del campo.

```
Mapped[Optional[int]]
```

Indica que puede ser un entero o `None`.

```
Integer
```

Tipo entero en base de datos.

```
nullable=True
```

Permite que el campo esté vacío.

Ejemplo conceptual:

```
severity_min = 3
```

La regla solo se cumpliría para eventos con severidad igual o superior a 3.

Si `severity_min` es `None`, la regla no filtra por severidad.

Campo `contains`

```
contains: Mapped[Optional[str]] = mapped_column(String(200), nullable=True)
```

Esta línea define la columna `contains`.

Permite indicar un texto que debe aparecer en el mensaje del evento.

Desglose:

```
contains
```

Nombre del campo.

```
Mapped[Optional[str]]
```

Puede ser una cadena o `None`.

```
String(200)
```

Cadena con longitud máxima de 200 caracteres.

```
nullable=True
```

Permite valores nulos.

Ejemplo:

```
contains = "failed login"
```

La regla podría aplicarse a eventos cuyo mensaje contenga ese texto.

Si `contains` es `None` , la regla no filtra por contenido del mensaje.

Comentario `throttle`

```
# throttle por regla (segundos)
```

Comentario que introduce el campo de `throttle`.

El `throttle` sirve para limitar la frecuencia con la que una regla genera alertas.

Esto evita generar demasiadas alertas repetidas en poco tiempo.

Campo `throttle_seconds`

```
throttle_seconds: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
```

Esta línea define la columna `throttle_seconds` .

Representa un periodo de espera, en segundos, entre alertas generadas por la misma regla.

Desglose:

```
throttle_seconds
```

Nombre del campo.

```
Mapped[Optional[int]]
```

Puede ser un entero o `None` .

```
Integer
```

Tipo entero.

```
nullable=True
```

Permite valores nulos.

Ejemplo conceptual:

```
throttle_seconds = 300
```

Esto podría significar que la regla no debe generar otra alerta hasta que pasen 300 segundos desde la última alerta similar.

Si es `None` , no se aplica `throttle`.

Comentario `threshold`

```
# threshold: N eventos en X segundos
```

Comentario que introduce los campos de `threshold`.

El `threshold` permite generar una alerta cuando se alcanza un número determinado de eventos dentro de una ventana temporal.

Ejemplo conceptual:

```
5 eventos en 60 segundos
```

Esto es útil para detectar patrones repetidos, como múltiples fallos de login en poco tiempo.

Campo `threshold_count`

```
threshold_count: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
```

Esta línea define la columna `threshold_count`.

Representa cuántos eventos deben producirse para que la regla se active.

Desglose:

```
threshold_count
```

Nombre del campo.

```
Mapped[Optional[int]]
```

Puede ser un entero o `None`.

```
Integer
```

Tipo entero.

```
nullable=True
```

Permite valores nulos.

Ejemplo:

```
threshold_count = 5
```

La regla requiere 5 eventos para activarse.

Campo `threshold_seconds`

```
threshold_seconds: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
```

Esta línea define la columna `threshold_seconds`.

Representa la ventana temporal del threshold.

Desglose:

```
threshold_seconds
```

Nombre del campo.

```
Mapped[Optional[int]]
```

Puede ser un entero o `None`.

```
Integer
```

Tipo entero.

```
nullable=True
```

Permite valores nulos.

Ejemplo:

```
threshold_seconds = 60
```

La regla evalúa si se han producido los eventos requeridos dentro de 60 segundos.

Combinado con `threshold_count`, permite expresar:

```
threshold_count = 5
threshold_seconds = 60
```

Es decir:

```
5 eventos en 60 segundos
```

Comentario `match por meta`

```
# match por meta
```

Comentario que introduce el campo `meta_match`.

Este campo permite definir condiciones sobre metadatos del evento.

Campo `meta_match`

```
meta_match: Mapped[Optional[dict[str, Any]]] = mapped_column(JSONB, nullable=True)
```

Esta línea define la columna `meta_match`.

Permite guardar un diccionario JSONB con condiciones sobre el campo `meta` de los eventos.

Desglose:

```
meta_match
```

Nombre del campo.

```
Mapped[Optional[dict[str, Any]]]
```

Indica que puede ser:

- Un diccionario con claves `string` y valores de cualquier tipo.
- `None`.

```
mapped_column(JSONB, nullable=True)
```

Define una columna PostgreSQL de tipo JSONB y permite valores nulos.

Ejemplo conceptual:

```
{
  "user": "admin",
  "action": "login_failed"
}
```

Una regla con ese `meta_match` podría buscar eventos cuyo campo `meta` contenga esos valores.

Campo `created_at`

```
created_at: Mapped[object] = mapped_column(
    DateTime(timezone=True),
```

```
server_default=func.now(),
nullable=False,
)
```

Este bloque define la columna `created_at`.

Representa la fecha de creación de la regla en la base de datos.

Está escrito en varias líneas para mejorar la legibilidad.

Nombre y tipo mapeado

```
created_at: Mapped[object]
```

Indica que `created_at` es una columna mapeada.

Se usa `object` como tipo genérico para el valor de fecha/hora.

Tipo de columna

```
DateTime(timezone=True)
```

Indica que la columna almacena fecha y hora con información de zona horaria.

Valor por defecto

```
server_default=func.now()
```

Indica que PostgreSQL asignará automáticamente la hora actual si no se proporciona un valor.

`func.now()` representa la función SQL `NOW()`.

Restricción de nulidad

```
nullable=False
```

Indica que el campo no puede ser nulo.

Toda regla debe tener una fecha de creación.

Resultado final del archivo

Después de cargar este archivo, SQLAlchemy dispone del modelo:

```
Rule
```

Este modelo representa la tabla:

```
rules
```

con los campos:

```
id
name
enabled
source
severity_min
contains
throttle_seconds
```

```
threshold_count
threshold_seconds
meta_match
created_at
```

7 Relación con el flujo técnico del laboratorio

El modelo `Rule` participa directamente en el motor de reglas del laboratorio.

La relación técnica sería:

```
Evento almacenado
    ↓
Consulta de reglas activas
    ↓
Rule.enabled = true
    ↓
Evaluación de criterios:
- source
- severity_min
- contains
- meta_match
    ↓
Evaluación de threshold/throttle si aplica
    ↓
Generación de alerta
```

Dentro del flujo general del SIEM:

```
Event
    ↓
se compara con
Rule
    ↓
si coincide, genera
Alert
```

Este modelo permite que el laboratorio tenga reglas configurables en lugar de condiciones fijas escritas directamente en el código.

8 Errores típicos o puntos importantes

`name` debe ser único

El campo:

```
unique=True
```

impide tener dos reglas con el mismo nombre.

Si se intenta crear una regla duplicada, PostgreSQL lanzará un error de restricción única.

`enabled` tiene valor por defecto

El campo:

```
server_default="true"
```

indica que una regla nueva estará activa por defecto si no se especifica lo contrario.

Los criterios pueden ser nulos

Campos como:

```
source
severity_min
contains
meta_match
```

son opcionales.

Esto permite crear reglas más generales o más específicas.

Por ejemplo:

```
Solo severity_min → regla amplia
source + contains → regla más concreta
meta_match → regla basada en metadatos
```

threshold_count y threshold_seconds funcionan como pareja

Para que una regla de threshold tenga sentido, normalmente deben existir ambos campos:

```
threshold_count
threshold_seconds
```

Ejemplo:

```
5 eventos en 60 segundos
```

Si solo existe uno de los dos, la lógica del motor de reglas debe decidir cómo tratar ese caso.

throttle_seconds evita ruido

El campo `throttle_seconds` permite evitar que la misma regla genere demasiadas alertas repetidas en poco tiempo.

Esto es importante en sistemas SIEM reales, donde una mala regla puede generar mucho ruido.

JSONB es específico de PostgreSQL

El campo:

```
meta_match
```

usa `JSONB`, que pertenece al dialecto PostgreSQL.

Esto refuerza que el proyecto está diseñado específicamente para PostgreSQL.

9 Comandos útiles relacionados

Comprobar que el modelo se puede importar:

```
docker exec -it siem-api python -c "from app.models.rule import Rule; print(Rule)"
```

Comprobar el nombre de la tabla:

```
docker exec -it siem-api python -c "from app.models.rule import Rule; print(Rule.__tablename__)"
```

Comprobar columnas del modelo:

```
docker exec -it siem-api python -c "from app.models.rule import Rule;
print(Rule.__table__.columns.keys())"
```

Comprobar estructura de la tabla `rules` :

```
docker exec -it siem-db psql -U siem -d siem -c "\d rules"
```

Consultar reglas existentes:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, enabled, source, severity_min,
contains, throttle_seconds, threshold_count, threshold_seconds, meta_match, created_at FROM rules LIMIT
10;"
```

Contar reglas totales:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM rules;"
```

Contar reglas activas:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM rules WHERE enabled IS TRUE;"
```

Consultar reglas con `threshold`:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, threshold_count, threshold_seconds
FROM rules WHERE threshold_count IS NOT NULL OR threshold_seconds IS NOT NULL;"
```

Consultar reglas con `throttle`:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, throttle_seconds FROM rules WHERE
throttle_seconds IS NOT NULL;"
```

Consultar reglas con `meta_match` :

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, meta_match FROM rules WHERE meta_match
IS NOT NULL;"
```